

CAHIER DES CHARGES RSA

I

Notre compréhension du projet et ses limites:

1.1 Cas général:

Le programme C va être simulation d'une autorité de certification décentralisée à l'aide d'une blockchain. En fait, si une blockchain de type Bitcoin est vraiment décentralisée c'est qu'elle est gérée par peer to peer.

Ici nous allons implémenter une blockchain centralisée mais on va faire comme si c'était du peer to peer

Le projet concerne la cryptographie asymétrique selon le système RSA (simplifié). Dans ce système il y a deux clés :

Une clé publique servant à envoyer un message chiffré à quelqu'un.

Une clé privée servant à ce quelqu'un de déchiffrer le message.

Le même système peut aussi servir à signer électroniquement un message avec une clé privée et à vérifier une signature avec la clé publique.

Le sujet sera découpé en plusieurs parties. Chaque partie sera constituée d'une explication et d'un cahier des charges de réalisation.

Il s'agit de programmer un système du type Gnu Privacy Guard (GPG).

1.1.2 Pour résumer:

Il faudra donc prendre conscience des fonctions mises à notre disposition puisqu'une bibliothèque de fonction est fournie (les outils de nombres premiers, factorisation, algorithme d'Euclide étendu (Bezout) ; et la création des clefs.) De plus, il faudra une bonne maîtrise du C du au manque de gestion automatique de dépassement de capacité. On sera limité à des uint64 soit des int positif codés sur 64 bit.

De plus, il est conseillé de faire une première lecture globale de tout le sujet pour comprendre où le sujet cherche à nous amener. Afin d'acquérir une vision globale du projet et donc ne pas avoir une vision de tunnel dès le début pour ne pas partir dans la mauvaise direction il faudra accompagner le projet d'un makefile pour la compilation et d'un README. Le clean code devra être implanté afin d'aider la relecture et de commenter les lignes les plus complexes

1.2 Liste des fichiers et fonctions déjà implanter:

— rsa_common_header.h contient les types utilisés dans rsa_tools.c. Vous pouvez tout changer mais il faudra adapter les fonctions de rsa_tools.c

— rsa_tools.c fonctions de génération de nombres aléatoires, tests de primalité, génération aléatoire de nombres premiers et de clefs.

— other_base64.c fonctions de conversion en base 64 et vice-versa.

— Bezout.c

— Sha-256 fonctions de hash pour les digests 4 / 10 Projet avancé septembre 2026 Licence Informatique

1. int premier (uint n) : test de primalité avec le crible d'Ératosthène, renvoie 1 si le nombre est premier, 0 sinon
2. uint random_uint(uint min,uint max) : génère un uint aléatoire entre min et max. min et max sont des uint, renvoie n : $\min \leq n \leq \max$
3. int décompose (uint facteur[], uint64 n) : décompose un entier en facteurs premiers (brute force), param[in] : n l'entier, param[out] : facteur = tableau de facteurs, returns la dimension du tableau
4. uint puissance(uint a, uint e) : puissance russe, param[in] : a l'entier et e l'exposant, returns : a^e
5. uint64 puissance_mod_n (uint64 a, uint64 e, uint64 n) : puissance modulaire, calcule $a^e \bmod n$, ATTENTION : $a \times a$ peut dépasser la capacité d'un uint64. Se référer à : [// https://www.labri.fr/perso/betrema/deug/poly/exp-rapide.html](https://www.labri.fr/perso/betrema/deug/poly/exp-rapide.html)
6. uint genereUint(uint max,int *cpt) : génère un nombre premier inférieur à max. param[in] max : la borne sup, param[out] cpt : nombre d'essais, returns : le nombre premier. cpt compte le nombre d'essais pour trouver le nombre premier.
7. int rabin (uint a, uint n) : test de Rabin sur la primalité d'un entier, c'est un test statistique
8. uint genereUintRabin(uint max,int *cpt) : fournit un nombre premier vérifié avec le test de Rabin
9. uint pgcdFast(uint a,uint b) : pgcd rapide de a et b, param[in] a et b les deux entiers, pgcd(a,b).
10. void genKeysRabin(rsaKey_t *pubKey,rsaKey_t *privKey) : génère une paire de clefs, param[out] pubKey : clef publique, param[out] privKey : clef privée
11. void inputKey(uint64 E,uint64 N,rsaKey_t *key) : pour debug : permet de saisir une clef
12. long bezoutRSA(uint a,uint b,long *u,long *v) : récupère (r,u,v) de Bézout. Si u est négatif on le remplace par le premier qui est supérieur à 2.
13. long bezout(uint a,uint b,long *u,long *v) : implémente l'algorithme d'Euclide étendu https://fr.wikipedia.org/wiki/Algorithme_d%27Euclide_%C3%A9tendu Fonction utilisée par la précédente. param[in] : a, b entiers (naturels), param[out] : u, v entiers relatifs tels que $r = \text{pgcd}(a, b)$ et $r = a * u + b * v$, returns r entier

II

Implémentation du chiffrement avancé et gestion des données

2.0 Implémentation des outils de base et tests:

Compréhension et affichage : Maîtriser les fonctions fournies et écrire une ou plusieurs fonctions pour afficher les clés générées au format hexadécimal.

Programme principal de test : Créer un fichier main dédié, à inclure dans le livrable, pour générer et afficher une paire de clés.

Gestion des fichiers de clés : Développer des fonctions permettant de sauvegarder une paire de clés dans un fichier local et de la recharger en mémoire dans le programme.

Validation du chiffrement unitaire : Tester le chiffrement et le déchiffrement sur un caractère UTF8 isolé, puis automatiser cette validation via une boucle utilisant plusieurs paires de clés différentes.

Analyse des limites matérielles : Modifier la constante MAXPRIME (initialement fixée à 10000 dans le fichier d'en-tête) afin de déterminer avec précision le seuil de dépassement de capacité du type uint64.

Ces tests de robustesse doivent figurer dans les livrables.

Documentation : Rédiger le fichier README incluant un bilan complet de cette première étape, en précisant la valeur limite trouvée pour MAXPRIME et en justifiant l'exhaustivité des tests réalisés.

2.1. Chiffrement et déchiffrement par tableau d'octets:

Principe et vulnérabilité : Implémenter un chiffrement et déchiffrement d'un message caractère par caractère stocké dans un tableau, en utilisant chaque octet du code UTF-8 comme un entier pour les calculs. Il faut prendre conscience que cette méthode est conceptuellement basique et reste très vulnérable aux attaques statistiques.

Fonction de chiffrement : Développer une fonction capable de chiffrer un tableau d'octets (uint8), dont le résultat retourné sera également un tableau d'octets (uint8).

Fonction de déchiffrement : Implémenter la fonction inverse permettant de déchiffrer un tableau d'octets (uint8) pour retrouver le message dans un nouveau tableau d'octets (uint8).

Politique de validation : Définir et mettre en place une politique de tests claire et rigoureuse pour valider le bon fonctionnement de ces deux fonctions.

PDF

